

3D Pool Game - Complete Documentation

Contents

SECTION 1 - Game Presentation	1
1.1 How to launch the game	1
1.2 How to play	2
1.3 Controls	2
1.4 Difficulty levels	2
1.5 Win conditions and level progression	2
1.6 Lives system	3
SECTION 2 - Technical Report	3
2.0 File overview and inter-script dependencies	3
2.1 index.html	3
2.2 source/main.js	4
Initialization order (init)	4
Runtime-managed elements (animate loop), in order	5
State machine (gameState)	5
Main functions of main.js	6
Notable constants	7
2.3 source/physics.js	7
2.4 source/controls.js	8
2.5 source/textures.js	9
2.6 source/sounds.js	9
2.7 source/models.js	10
Hierarchies built procedurally in models.js	11
2.8 libs/ (third-party libraries)	11

SECTION 1 - Game Presentation

This is a 3D pool (billiards) game played in the browser. The scene depicts a pool table in a furnished night-time room. The goal is to pocket every colored ball by striking them with the white cue ball.

1.1 How to launch the game

The game is playable online at the project's GitHub Pages [link](#).

No installation is required, just open the link in a browser.

To run it locally, serve the project folder with any static file server (e.g. `python -m http.server 7777`) and open the printed URL; opening `index.html` directly from disk does not work.

On startup, a loading screen appears, followed by a difficulty-selection menu, after which the match begins.

1.2 How to play

The mechanic is that of pool: strike the white cue ball to pocket the colored balls.

A shot is a single press-and-release gesture: drag to aim the cue, hold still to charge power, and release to take the shot. Dragging cancels any charge in progress, so releasing right after a drag does not fire a shot. The cue cannot be aimed through another ball - it automatically adjusts to graze around it instead.

If the cue ball itself is accidentally pocketed, it respawns on the table after a brief moment.

Buttons are available to rearrange the balls, reset the game, switch the camera view, toggle the table lamp and ceiling light, and toggle the music.

1.3 Controls

Control	Action
Drag	Aim the cue
Hold (press and hold)	Charge power
Release	Take the shot
Mouse wheel / scroll	Zoom (Player POV view only)
L	Toggle the table lamp
M	Toggle music
C or V	Switch view (Overview ↔ Player POV)
R	Reset the game
N	New ball arrangement
O	Toggle the ceiling light

All of the above are also available as on-screen buttons. The controls work with touch as well: touch-and-hold to charge, drag to aim, release to shoot.

1.4 Difficulty levels

Before starting, a menu lets you choose a difficulty: Normal, Hard, or Insane. The difficulties differ in the number of lives you're given. Insane is the most punishing, with a single life, then you have three in medium and five in three.

The number and arrangement of balls is the same across all difficulties.

1.5 Win conditions and level progression

The game has 4 levels. Each level requires pocketing an increasing number of colored balls. You win the game if you finish all the levels.

1.6 Lives system

Lives are your attempts, shown as hearts. You lose a life whenever a shot fails to pocket at least one colored ball (sinking only the cue ball does not count). Losing a life does not reset your progress in the current level, but when lives run out, “GAME OVER”!

SECTION 2 - Technical Report

2.0 File overview and inter-script dependencies

The project is a WebGL application built on **Three.js r128** (vendored locally). The HTML entry point is `index.html`, which loads `source/main.js` as an ES module. All application scripts live in `source/`.

```
index.html
  '-- <script type="module" src="source/main.js">
    main.js -- imports --> models.js    (mesh/hierarchy construction + glTF/GLB loading)
              -- imports --> textures.js (procedural texture generation + PBR texture loading)
              -- imports --> physics.js  (pure physics simulation, no Three.js)
              -- imports --> controls.js  (mouse/touch input: aim, charge, shoot)
              -- imports --> sounds.js    (audio management: music and effects)

    models.js -- imports --> physics.js  (constants TABLE_W, TABLE_H, BALL_RADIUS, POCKET_POSITIONS)
              -- imports --> three/addons/loaders/GLTFLoader.js
```

Concise dependency map (who imports what):

Script	Imports from	Imported by
main.js	models.js, textures.js, physics.js, controls.js, sounds.js, three	(entry point, none)
models.js	three, GLTFLoader, physics.js	main.js
physics.js	(none)	main.js, models.js
controls.js	(none)	main.js
textures.js	three	main.js
sounds.js	(none)	main.js

`physics.js` and `controls.js` are deliberately decoupled from `Three.js`: they operate on plain data objects (the ball state and the input state) that `main.js` reads every frame.

The `importmap` in `index.html` resolves `three` to `./libs/three.module.js` and `three/addons/loaders/GLTFLoader.js` to `./libs/GLTFLoader.js`.

2.1 index.html

Entry page. It contains the full-viewport `<canvas id="glCanvas">`, the `Three.js` module `importmap`, all of the UI's CSS (HUD, overlays, power bar, buttons, legend, life-loss animations), and the static UI elements. It loads `source/main.js` as an ES module.

Logical sub-sections of the file:

- **Importmap:** maps `three` → `./libs/three.module.js` and the `GLTFLoader` → `./libs/GLTFLoader.js`.

- **Styles / CSS animations:** keyframes for overlays (`bounceIn`, `spinBall`, `floatTrophy`, `pulseGlow`, `starPop`, `confettiFall`), for life loss (`heartLose`, `hudShake`, `screenTremble`, `lifeFlash`, `heartFragment`, `heartFlyAndExplode`, `heartPulse`), and for the power-bar tremble (`tremble`). Dedicated overrides for the “insane” theme (`body.difficulty-insane ...`).
- **Static DOM elements:** `#glCanvas`, `#loading-overlay`, `#hud` (with `#hud-level`, `#hud-balls`, `#hud-lives`), `#overlay`, `#bottom-ui` (with `#power-bar/#power-fill` and the button bar `#btn-newconfig`, `#btn-reset`, `#btn-cam`, `#btn-lamp`, `#btn-ceiling`, `#btn-music`), `#legend`.

2.2 source/main.js

This is the main script: engine bootstrap (renderer, scene, cameras), render loop, game loop, state machine, level management, lives, difficulty, HUD, and overlays.

Initialization order (init)

`init()` is registered on `DOMContentLoaded` and executes, in this exact order:

1. **Loading manager:** registers `THREE.DefaultLoadingManager.onLoad/onError` to track the loading of all textures/models; sets a 15 s safety-net timeout that force-closes the loading overlay.
2. **Audio:** `initSounds()`.
3. **Renderer:** creates the `THREE.WebGLRenderer` (antialias, `PCFSoftShadowMap` shadows, `sRGBEncoding`, `ACESFilmic` tone mapping, exposure 0.75, night-sky clear color) and performs an initial `clear()`.
4. **Scene:** creates the `Scene`, sets a procedural night-sky background (`_buildNightSkyBackground`) and fog (`Fog`).
5. **Cameras:** `camera0` (Overview, 52° FOV) and `camera1` (Player POV, 58° FOV); `activeCamera = camera0`.
6. **Lights:** `HemisphereLight`, `AmbientLight`, `DirectionalLight` (intensity 0).
7. **Textures:** `texMap = generateTextures()`.
8. **Scene geometry:** in order - `createRoom`, `createTable`, `createLamp`, `createLoungeCorner`, `createDartboard`, `createCabinet`, `createStools`, `createPainting`, `createFrame2`, `createPlant`, `createPlant2`, `createCoatRack`, `createPainting3`, `createPlant2Corner`, `createFloorLamp`, `createCeilingLight`. The ceiling light is initialized off (intensity 0, fixture hidden).
9. **Cue stick:** `createCueStick`.
10. **Controls:** `new Controls(canvas)`.
11. **Environment map:** `_buildEnvMap()` (PMREM) and `scene.environment`.
12. **Clock:** `new THREE.Clock()`.
13. **HUD/UI:** `_buildHUD()`, `_bindUIButtons()`.
14. **Events:** `resize`, `keydown`, `wheel`.
15. **Level start:** `_startLevel(0)`.

16. **Render loop:** `animate()`; a subsequent `requestAnimationFrame` sets `_framePainted = true` and attempts to dismiss the loading overlay.

The loading overlay is dismissed by `_dismissLoader()` when both `_resourcesReady` (onLoad fired) and `_framePainted` (one frame rendered) are true; on dismissal it shows the difficulty menu (`_showDifficultyMenu`).

Runtime-managed elements (animate loop), in order

Each frame `animate()` performs: (1) delta-time computation (capped at `DT_CAP`);

(2) **input** (`controls.update()`, `consumeShot()` → possible `_fireShot`); (3) **physics** (only in the ROLLING/STRIKING states: adaptive sub-stepping of 1 or 3 steps via `stepPhysics`, starting a pocket-drop animation for each newly-pocketed ball, cue-ball respawn, counter decrement, level-complete check, and when the table is at rest `snapToRest` + possible `_loseLife`); (4) **synchronization** of the ball meshes with the physics state plus rolling-rotation computation; (4b) **pocket-drop animation** update (`_updatePocketing`) for any ball still falling into a hole; (5) **cue** (`_updateCue`); (6) **lamp** (swing, `_updateLamp`); (7) **camera** (Player POV update if active); (8) **render** + power-bar update.

State machine (gameState)

The STATE object defines three states:

State	Meaning	What happens
WAITING	Waiting for the shot	The cue follows the cursor/aim, the charge bar is live, a shot may fire. Controls are enabled.
STRIKING	Strike in progress	The cue snaps forward through its short animation (<code>STRIKE_FORWARD_TIME</code>); when the tip reaches the ball surface the cue-ball velocity is applied and the impact sound plays; controls are disabled.
ROLLING	Balls in motion	Physics is integrated every frame, the cue is hidden; a pocketed ball animates falling into its hole independently of this check; once every remaining ball “looks stopped” it returns to WAITING.

Transitions:

- WAITING → STRIKING: in `animate`, when `consumeShot()` returns a shot (`_fireShot` sets `gameState = STRIKING`, disables controls, records the pending angle/power).
- STRIKING → ROLLING: in `_updateCue`, when `strikeTimer >= STRIKE_SHOW_TIME` (hides the cue).
- ROLLING → WAITING: in `animate`, when `isReadyForNextShot(balls)` is true and the cue ball is not pocketed (`snapToRest`, re-enables controls, re-shows the cue; if no colored ball was pocketed that shot it calls `_loseLife`).
- Early ROLLING/STRIKING → WAITING: when the last colored ball is pocketed (`ballsRemaining <= 0`), it sets WAITING and schedules `_onLevelComplete`.

Additional states not represented in `gameState` but handled via flags:

`gameWon`, `gameOver`, `difficultyChosen` (they control the win overlay, game over, and the difficulty menu).

Main functions of `main.js`

Name	Description (brief)
<code>init</code>	Full bootstrap: renderer, scene, cameras, lights, geometry, controls, HUD, events, loop start.
<code>_maybeDismissLoader</code> / <code>_dismissLoader</code>	Dismiss the loading overlay once resources and the first frame are ready; idempotent.
<code>_buildNightSkyBackground</code>	Generates, via canvas, the equirectangular night-sky texture (stars, moon, halo) used as the background.
<code>_buildEnvMap</code>	Creates a PMREM environment map from a canvas gradient for the balls' specular reflections.
<code>_onResize</code>	Updates the cameras' aspect ratio and the renderer size on window resize.
<code>_onKeyDown</code>	Maps the keys L/O/C/V/R/N/M to their actions (lights, camera, reset, new config, music).
<code>_onWheel</code>	Zooms the Player POV camera by varying <code>camDistBehind</code> (clamped 1.5–6.0).
<code>_startLevel</code>	Starts/restarts a level: removes the old balls, spawns new ones, resets the cue and state → WAITING.
<code>_spawnBall</code>	Creates a ball mesh (color/number) and registers its physics state in the <code>balls</code> array.
<code>_newConfiguration</code>	Re-arranges the current level's balls ("Rearrange" button / N key), if the state allows.
<code>_resetGame</code>	Cancels pending transitions and returns to the difficulty menu ("Reset" button / R key).
<code>_respawnCueBall</code>	Repositions the cue ball after a "scratch" (accidental pocketing).
<code>_fireShot</code>	Starts the strike: records the pending angle and power and switches to the STRIKING state.
<code>_wrapPi</code>	Wraps an angle to the $(-\pi, \pi]$ range.
<code>_resolveAimAngle</code>	Computes a collision-free aim angle, making the cue "glide" around the other balls.
<code>_updateCue</code>	Updates the cue's position/rotation per state; in STRIKING applies the velocity to the cue ball on contact.
<code>_updateBallRotation</code>	Applies rolling rotation to a ball's mesh based on its velocity.
<code>_nearestPocketCenter</code>	Finds the centre of the pocket nearest a point - the hole a captured ball is dropping into.
<code>_startPocketDrop</code>	Begins a captured ball's fall-into-the-pocket animation (instead of hiding it instantly), scaling fall speed, funnelling, and tumble rate to how fast the ball arrived.
<code>_updatePocketing</code>	Per-frame update of every in-flight pocket drop: applies gravity, eases the ball toward the hole centre, tumbles it, times the drop sound, and removes/hides it once fully sunk.
<code>_updateLamp</code>	Swings the pendant lamp (sinusoidal rotation of the anchor).
<code>_toggleMusic</code> / <code>_toggleLamp</code> / <code>_toggleCeiling</code>	Toggle music, the table lamp, and the ceiling light respectively, updating the HUD.
<code>_switchCamera</code>	Alternates between the Overview and Player POV cameras; shows/hides the ceiling fixture accordingly.
<code>_updatePlayerCamera</code>	Positions the Player POV camera behind the cue ball, based on the cue's angle.
<code>animate</code>	Main render loop: timing, input, physics, mesh sync, cue, lamp, camera, render.

Name	Description (brief)
<code>_onLevelComplete</code>	Decides whether to show the level-complete screen (and advance) or the final win screen.
<code>_showLevelComplete</code>	Shows the “Level Complete” overlay with stars and a quip; plays the success sound.
<code>_showWinScreen</code>	Shows the final “YOU WIN!” overlay with confetti and a “Play Again” button.
<code>_buildHUD</code>	Caches the HUD’s DOM references and creates the life-loss flash element.
<code>_bindUIButtons</code>	Binds the UI buttons to their actions and the click sound.
<code>_updateHUD</code>	Updates the level text, balls remaining, button labels, and the lives hearts.
<code>_updatePowerBar</code>	Updates the power bar’s width/color/glow based on the charge.
<code>_showDifficultyMenu</code>	Builds and shows the difficulty-selection menu with the slider and animated SVG face.
<code>_selectDifficulty</code>	Applies the chosen difficulty (lives, theme, music rate) and starts Level 1.
<code>_loseLife</code>	Handles losing a life with animations (different for “insane”) and the eventual game over.
<code>_spawnHeartFragments</code>	Spawns the animated fragments (broken heart / purple heart / sparkle emoji) when the heart explodes.
<code>_showGameOver</code>	Shows the “GAME OVER” overlay, stops the music, and offers “Try Again”.

Notable constants

`LEVELS_BALL_COUNT` = [1,2,3,4] (balls per level), `NUM_LEVELS` = 4, lamp swing parameters, POV camera distance/height, strike timings, pocket-drop animation tuning (`POCKET_GRAVITY`, `POCKET_SINK_EASE`, `POCKET_REST_Y`, `POCKET_DROP_V0`, `POCKET_SPEED_REF`), `BALL_COLORS` (palette with the cue ball + 8 solid colors; the 9–15 entries for the striped balls are present but **commented out**, hence inactive).

2.3 source/physics.js

Pure-JavaScript physics simulation, with **no Three.js dependency**. It operates on ball-state objects { `id`, `isCueBall`, `x`, `z`, `vx`, `vz`, `pocketed`, `mesh` } in the XZ plane. When a ball is captured, `stepPhysics` also records its arrival speed as `pocketSpeed`, which `main.js` reads to scale the pocket-drop animation.

Logical sub-sections:

- **Tunable constants:** table dimensions (`TABLE_W` = 9.0, `TABLE_H` = 4.5), `BALL_RADIUS` = 0.18, `POCKET_RADIUS` = 0.32, friction (`FRICTION_60FPS`), cushion/ball restitution, the minimum-speed and “ready for next shot” thresholds.
- **Game geometry:** the cushions’ inner edges, the positions of the 6 pockets (`POCKET_POSITIONS`: 4 corners + 2 sides), the exclusion zones around the pockets.
- **Pocket / cushion / ball-ball collision detection.**
- **Main step and state utilities.**

Name	Description (brief)
<code>_isInPocket</code>	Tells whether a ball center is inside a pocket’s capture zone (direct capture or “throat” capture).
<code>_nearPocketOpening</code>	Tells whether a ball is close enough to a pocket to suppress the cushion bounce.
<code>_resolveCushion</code>	Reflects velocity off the rectangular boundaries and clamps the ball’s position; skips the check near pockets.

Name	Description (brief)
<code>_resolveBallBall</code>	Resolves the elastic collision between two equal-mass balls (overlap correction + impulse).
<code>stepPhysics</code>	Advances the simulation by one <code>dt</code> : integrates positions, applies friction, resolves collisions, detects pocketing (recording each captured ball's arrival speed as <code>pocketSpeed</code>); returns the newly pocketed balls.
<code>isReadyForNextShot</code>	True once every non-pocketed ball's speed is below the perceptual "looks stopped" floor.
<code>snapToRest</code>	Zeroes <code>vx/vz</code> on every non-pocketed ball (called when leaving ROLLING).
<code>randomizeBalls</code>	Generates valid, non-overlapping start positions (index 0 = cue ball on the player side; the rest = colored).

`stepPhysics` takes optional callbacks (`onBallBall`, `onCushion`) used by `main.js` to play impact sounds based on velocity. Ball-ball collision is $O(n^2)$, which is adequate for the small number of balls.

2.4 source/controls.js

Mouse/touch input management for aiming, charging, and shooting. No Three.js dependency: it exposes public state that `main.js` reads every frame.

Gesture model (one press-and-release on the canvas): holding still charges power (release = shot); dragging past a tolerance switches the gesture into "aim" and cancels the charge.

Public state exposed by the `Controls` class: `aimAngle`, `mouseX/mouseY`, `chargeAmount` (0–1), `pullback`, `enabled`, and the getters `isCharging/isAiming`.

Name	Description (brief)
<code>constructor</code>	Initializes the public and internal state and binds the listeners to the canvas.
<code>_bindEvents</code>	Registers the mouse (down/move/up/leave) and touch (start/move/end) listeners and disables the context menu.
<code>_onMouseDown</code>	Begins a press: starts charging and records the start point of the possible drag.
<code>_onMouseMove</code>	Updates the cursor position; past the drag tolerance it switches into aim mode and rotates <code>aimAngle</code> .
<code>_onMouseUp</code>	Ends the press: if it was still charging it computes power from the hold duration and queues the shot.
<code>_onMouseLeave</code>	Aborts the current press without firing (the cursor leaves the canvas).
<code>_onTouchStart</code>	/ Touch versions of the mouse handlers, with <code>preventDefault</code> .
<code>_onTouchMove</code>	
<code>_onTouchEnd</code>	
<code>update</code>	Recomputes <code>chargeAmount</code> and <code>pullback</code> from the press duration (called every frame).
<code>consumeShot</code>	Returns the pending shot and clears it, so each shot is consumed exactly once.

Constants: `AIM_SENSITIVITY`, `MAX_CHARGE_TIME` = 2.5 s, `MAX_POWER` = 14.0, `MIN_POWER` = 0.5, `MAX_PULLBACK` = 1.4, the mouse/touch drag tolerances. Shot power is `MIN_POWER + (charge fraction) x (MAX_POWER - MIN_POWER)`.

2.5 source/textures.js

Procedural (canvas) texture generation and loading of PBR texture sets from files. No geometry or scene code.

Sub-sections / surfaces handled:

- **Felt (table surface) - procedural:** color (green baize with fiber noise), normal map (high-frequency, Sobel-like height field), roughness map.
- **Wood (rails/legs) - PBR files:** textures/wood/color.jpg, normal.jpg, roughness.jpg (repeat 6×1).
- **Leg (table legs) - PBR files:** loads textures/wood/color.jpg and roughness.jpg separately with a 1×2 repeat.
- **Floor - PBR files:** textures/floor/color.jpg, normal.jpg, roughness.jpg, ao.jpg (repeat 10×8).
- **Wall - PBR files:** textures/wall/color.jpg, normalgl.jpg, roughness.jpg (repeat 4×2).
- **Wood020 (window frame + skirting boards) - PBR files:** textures/wood020/color.jpg, normal.jpg, roughness.jpg (repeat 3×1).
- **Ball - procedural:** a per-ball texture factory + one shared roughness map.

Name	Description (brief)
<code>generateTextures</code>	Creates/loads every texture and returns the <code>texMap</code> object with the felt, wood, leg, wood020, floor, wall, ball entries and the <code>createBallTex</code> factory.
<code>_createFeltColorTexture</code>	Generates the felt color texture (green base + fiber speckling + fine weave grain).
<code>_createFeltNormalMap</code>	Generates the felt normal map from a high-frequency height field.
<code>_createFeltRoughnessMap</code>	Generates a mostly-rough felt roughness map with slight variation.
<code>_createBallTexture</code>	Generates a ball's texture: base color, optional white band (balls 9–15), and a number label (cue ball excluded).
<code>_createBallRoughnessMap</code>	Generates the shared ball roughness map (glossy resin with rare “scuff” specks).

Note: the band for the striped balls (numbers 9–15) is implemented in `_createBallTexture`, but the game only uses balls 0–4 (see `BALL_COLORS` and `LEVELS_BALL_COUNT`), so that branch is never triggered during play.

2.6 source/sounds.js

Audio management: one background music track (an `Audio` element) and the sound effects played through the Web Audio API (`AudioContext` + decoded buffers). The file paths are resolved relative to the module (`../sounds/`).

Effects loaded: `hitEffect.mp3`, `ballHit.mp3`, `ballWall.mp3`, `ballDrop.mp3`, `success.mp3`, `fail.mp3`, `win.mp3`, `error.mp3`, `heartBroken.mp3`, `click.mp3`; music: `background.mp3` (looped, volume 0.15).

Name	Description (brief)
<code>initSounds</code>	Initializes the music and the <code>AudioContext</code> , sets up audio “unlocking” on the first user gesture, and loads all buffers.
<code>_loadBuffer</code>	Fetches and decodes an audio file into an <code>AudioBuffer</code> (returns null on error).
<code>startBgMusic</code> /	Start/stop the background music (with resume on user gesture if blocked).
<code>stopBgMusic</code>	
<code>setMusicRate</code>	Sets the music playback rate.
<code>setMusicDifficulty</code>	Sets the music rate based on difficulty (1.4× for “insane”, otherwise 1.0×).
<code>_playBuffer</code>	Plays a buffer with gain proportional to the passed speed, waiting for the context to be running.
<code>playBallHitSound</code> /	Effects for ball-ball collision, ball-cushion, and pocketing.
<code>playBallWallSound</code> /	
<code>playBallDropSound</code>	
<code>playErrorSound</code> /	Effects for error, level cleared, game over, win, heart break, and button clicks.
<code>playSuccessSound</code> /	
<code>playFailSound</code> /	
<code>playWinSound</code> /	
<code>playHeartBrokenSound</code>	
<code>/ playClickSound</code>	
<code>playHitSound</code>	The cue-strike effect, with gain proportional to the shot power.

2.7 source/models.js

Construction of Three.js meshes, hierarchies, and materials. No game logic. It imports from `physics.js` the constants `TABLE_W`, `TABLE_H`, `BALL_RADIUS`, `POCKET_POSITIONS`. Y-up coordinate system: floor at `Y=0`, table surface at `TABLE_SURFACE_Y = 0.76`.

Exported constants: `TABLE_SURFACE_Y`, `BALL_Y` (resting ball center), `CUE_REACH` (cue length, used by `main.js` for the aim collision), `CUE_CLEAR_R` (the cue’s effective half-thickness).

Name (export)	Description (brief)
<code>createRoom</code>	Builds the closed room: floor, ceiling, 4 walls (box rendered from the inside), skirting boards, a window with hole/glass/frame/moonlight.
<code>createTable</code>	Builds the table: felt, wooden body, rails/cushions, 6 pockets, 4 tapered-box legs (hand-built <code>BufferGeometry</code>).
<code>_buildRails</code>	Builds the four rail/cushion segments, leaving gaps for the pockets.
<code>_buildPockets</code>	Builds, for each pocket, a tapered open cylinder with a baked vertex-color depth gradient (lit, so the holes brighten or dim with the table lamp and ceiling light) plus a black bottom cap.
<code>createCueStick</code>	Creates the cue stick as a parent-child hierarchy (root → group → tip/shaft/grip).
<code>createBallMesh</code>	Creates a ball mesh with <code>MeshPhysicalMaterial</code> (glossy clearcoat) and a per-ball texture.
<code>createLamp</code>	Creates the pendant lamp: a horizontal bar on two chains with three shades, emissive bulbs, and shadow-casting <code>SpotLights</code> .
<code>_createChainLinks</code>	Builds a vertical chain as a stack of alternating links (torus rings).
<code>createFloorLamp</code>	Creates a floor lamp in the lounge corner (base, pole, conical shade, bulb, warm <code>PointLight</code>).

Name (export)	Description (brief)
<code>createCeilingLight</code>	Creates the toggleable ceiling fixture (emissive lens + ring + shadow-casting <code>PointLight</code>).

Hierarchies built procedurally in `models.js`

Cue stick (`createCueStick`) - parent-child hierarchy:

```
cueRoot (Object3D, pivot at the cue-ball center; Y rotation = aim)
'-- cueGroup (Object3D, local X translation for pullback/strike)
  |-- tipMesh (cylinder, reddish-brown leather tip 0xcc4400)
  |-- shaftMesh (tapered cylinder, blonde wood 0xf5e8c0)
  '-- gripMesh (tapered cylinder, dark mahogany 0x3b1a0a)
```

All sections extend along the local +X axis; there are no keyframe animations (the cue’s “snap” is animated procedurally in `main.js`, not with animation clips).

Pendant lamp (`createLamp`) - hierarchy (everything a child of the anchor, which rotates to swing):

```
anchor (Group, pivot at the ceiling)
|-- 2 x mount (cylinder, brass) - on the rotation axis
|-- 2 x chain (Group of 8 alternating torus links)
|-- bar (horizontal cylinder, brass)
|-- 2 x finial (decorative sphere at the ends)
'-- for each of the 3 shades (x = -, 0, +):
  |-- socket (cylinder)
  |-- shade (outer green hemisphere)
  |-- liner (inner emissive white hemisphere)
  |-- trim (gold torus ring)
  |-- bulb (emissive sphere)
  |-- SpotLight (downward cone, shadow-casting)
  '-- target (Object3D, the SpotLight’s target)
```

Lamp materials: `goldMat` (brass), `shadeOuterMat` (baize green), `shadeInnerMat` (emissive white), `bulbMat` (emissive amber). `createLamp` returns { `anchor`, `bulbMeshes`, `linerMeshes`, `lights` }, used by the lamp toggle in `main.js`.

Table (`createTable`): felt (`PlaneGeometry`), body (`BoxGeometry`), rails (4 boxes via `_buildRails`), pockets (vertex-colored tapered cylinder + cap via `_buildPockets`), 4 legs (tapered box built from a hand-built `BufferGeometry`). Materials from `texMap`: felt, wood (body/rails), leg (legs).

Room (`createRoom`): room box with inward-facing faces, a separate floor (`PlaneGeometry` with `uv2` for AO), skirting boards, a window rebuilt with 4 holed panels + 4 “reveal” faces + glass (`MeshPhysicalMaterial`) + wooden frame, a `PointLight` of moonlight.

Balls (`createBallMesh`): `SphereGeometry`(`BALL_RADIUS`, 32, 32) with `MeshPhysicalMaterial` (roughness 0.15, clearcoat 1.0, `envMap`). The color/number texture is generated per-instance by `createBallTex`.

2.8 libs/ (third-party libraries)

The `libs/` folder contains the third-party dependencies vendored locally, which are **not written as part of the project**:

- `three.module.js` - the Three.js library (r128), the WebGL rendering engine used by all scripts.
- `GLTFLoader.js` - Three.js's official glTF/GLB loader, used in `models.js` to import 3D model files.

These files are not documented function by function because they are external code; their use within the project is described in the previous sections (importmap in `index.html`, imports in `main.js/models.js`).